

Lab 6.2 Database & Visualization - Python MySQL Hands-on Activities

2.1 Python Programming Using PyMySQL

The example sample code ([lab6_sample.py](#)) is on Github repo and as below. This example is generating 3 random values and then executing INSERT statements to store the values on the MySQL database. The sampling period is 3 seconds and total collection time is 60 seconds. Please go through the code and comments on each line to understand program logic. Please note that you need to modify the 'USER', 'PASSWORD', and 'TABLE' lines to run the sample script.

Python - Python3 ([lab6_sample.py](#))

```
import datetime
import time
import random
import pymysql.cursors

## Credential
HOST = 'mepotrb16.ecn.purdue.edu' # MySQL server host DNS
PORT = 3306 # MySQL server port number
USER = 'yourname' # MySQL account name
PASSWORD = 'password' # Password of the account
DB = 'ME597Spring24' # DB name (you should double check your database name)
TABLE = 'yourname_lab6' # table name
## Credential

sensor = 'sensor3' # sensor name
measurement1 = 'acceleration1' # measurement name 1
measurement2 = 'acceleration2' # measurement name 2
measurement3 = 'acceleration3' # measurement name 3

connection = pymysql.connect(host=HOST, user=USER, password=PASSWORD, db=DB, port=PORT) #
make a connection to MySQL server
cursor = connection.cursor() # Open cursur to execute SQL query

duration = 60 # collection time in second unit
start_time = time.time() # get current time

while time.time() - start_time < duration:
    value1 = str(random.uniform(0,2)) # random value for measurement1
    value2 = str(random.uniform(-2,0)) # random value for measurement2
    value3 = str(random.uniform(9.7,9.9)) # random value for measurement3
    timestamp = datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S') # get datetime as
MySQL timestamp format

    print(timestamp)
    print("{}={}".format(measurement1,value1)) # value1
    print("{}={}".format(measurement2,value2)) # value2
    print("{}={}".format(measurement3,value3)) # value2

    query1 = "INSERT INTO "+TABLE+" (timestamp,sensor,measurement,value)
VALUE('"+timestamp+"','"+sensor+"','"+measurement1+"','"+value1+"');" #SQL query1 for
value1
    query2 = "INSERT INTO "+TABLE+" (timestamp,sensor,measurement,value)
VALUE('"+timestamp+"','"+sensor+"','"+measurement2+"','"+value2+"');" #SQL query2 for
value2
```

```

query3 = "INSERT INTO "+TABLE+" (timestamp,sensor,measurement,value)
VALUE('"+timestamp+"','"+sensor+"','"+measurement3+"','"+value3+"');" #SQL query3 for
value3

cursor.execute(query1) # execute query 1
cursor.execute(query2) # execute query 2
cursor.execute(query3) # execute query 3
connection.commit() # commit all SQL queries

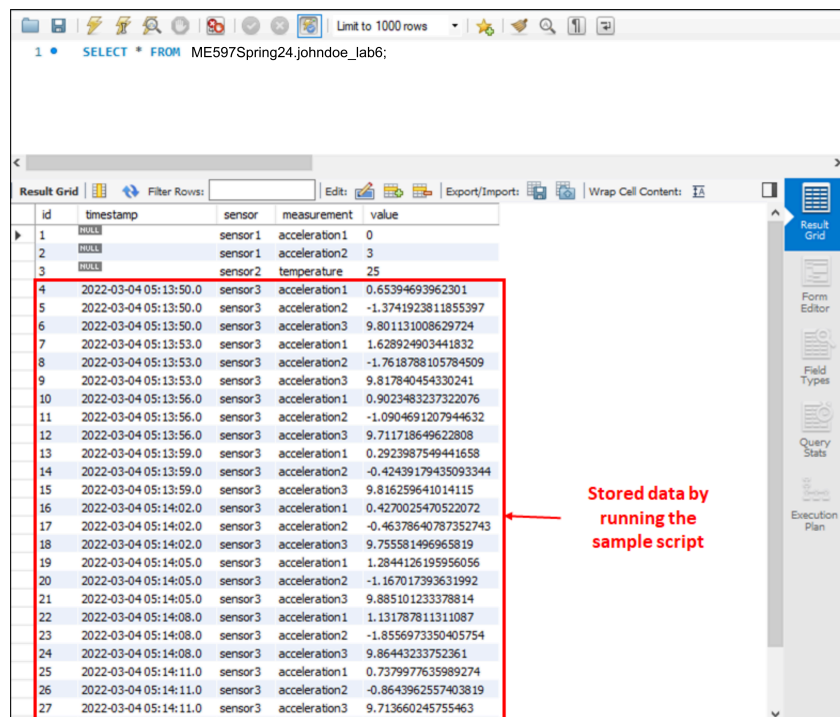
print("==INSERT QUERIES DONE==\n")

time.sleep(3) # wait for 3 seconds

connection.close() # gently close connection
print("==Program DONE==")

```

After run the sample code, you can see the stored data on 'MySQL Workbench' on laptop as Figure 5.



	id	timestamp	sensor	measurement	value
1	NULL		sensor1	acceleration1	0
2	NULL		sensor1	acceleration2	3
3	NULL		sensor2	temperature	25
4		2022-03-04 05:13:50.0	sensor3	acceleration1	0.65394693962301
5		2022-03-04 05:13:50.0	sensor3	acceleration2	-1.3741923811855397
6		2022-03-04 05:13:50.0	sensor3	acceleration3	9.801131008629724
7		2022-03-04 05:13:53.0	sensor3	acceleration1	1.628924903441832
8		2022-03-04 05:13:53.0	sensor3	acceleration2	-1.7618788105784509
9		2022-03-04 05:13:53.0	sensor3	acceleration3	9.817840454330241
10		2022-03-04 05:13:56.0	sensor3	acceleration1	0.9023483237322076
11		2022-03-04 05:13:56.0	sensor3	acceleration2	-1.0904691207944632
12		2022-03-04 05:13:56.0	sensor3	acceleration3	9.711718649622808
13		2022-03-04 05:13:59.0	sensor3	acceleration1	0.2923987549441658
14		2022-03-04 05:13:59.0	sensor3	acceleration2	-0.42439179435093344
15		2022-03-04 05:13:59.0	sensor3	acceleration3	9.816259641014115
16		2022-03-04 05:14:02.0	sensor3	acceleration1	0.4270025470522072
17		2022-03-04 05:14:02.0	sensor3	acceleration2	-0.46378640787352743
18		2022-03-04 05:14:02.0	sensor3	acceleration3	9.755581496965819
19		2022-03-04 05:14:05.0	sensor3	acceleration1	1.2844126195956056
20		2022-03-04 05:14:05.0	sensor3	acceleration2	-1.167017393631992
21		2022-03-04 05:14:05.0	sensor3	acceleration3	9.885101233378814
22		2022-03-04 05:14:08.0	sensor3	acceleration1	1.131787811311087
23		2022-03-04 05:14:08.0	sensor3	acceleration2	-1.8556973350405754
24		2022-03-04 05:14:08.0	sensor3	acceleration3	9.86443233752361
25		2022-03-04 05:14:11.0	sensor3	acceleration1	0.7379977635989274
26		2022-03-04 05:14:11.0	sensor3	acceleration2	-0.8643962557403819
27		2022-03-04 05:14:11.0	sensor3	acceleration3	9.713660245755463

Figure 5 MySQL Workbench result grid after running sample script

Task 2.1

Run the sample code on Raspberry Pi. Capture the result grid on MySQL workbench as Figure 5 and attach it to the report.

MySQL Workbench

ME 597 IoT x ME 597 IoT x

File Edit View Query Database Server Tools Scripting Help

Navigator

Filter objects

ME597Spring260

Tables

grafana_sample

robertclaud_lab6

Views

Stored Procedures

Functions

Query 1 robertclaud_lab6

Limit to 1000 rows

1 • SELECT * FROM ME597Spring260.robertclaud_lab6;

Result Grid

id	timestamp	sensor	measurement	value
1	2026-03-03 01:34:25.0	sensor 1	acceleration1	0
2	2026-03-03 01:34:25.0	sensor 1	acceleration2	3
3	2026-03-03 01:34:25.0	sensor 2	temperature	25
4	2026-03-03 01:34:25.0	sensor 3	acceleration1	1.658703299728456
5	2026-03-03 01:34:25.0	sensor 3	acceleration2	-0.5506682353415508
6	2026-03-03 01:34:25.0	sensor 3	acceleration3	9.749601227334084
7	2026-03-03 01:34:28.0	sensor 3	acceleration1	0.12359795624719427
8	2026-03-03 01:34:28.0	sensor 3	acceleration2	-1.7774015588705177
9	2026-03-03 01:34:28.0	sensor 3	acceleration3	9.791077127381895
10	2026-03-03 01:34:32.0	sensor 3	acceleration1	1.489193752207377
11	2026-03-03 01:34:32.0	sensor 3	acceleration2	-1.3720548103679873
12	2026-03-03 01:34:32.0	sensor 3	acceleration3	9.88810692825377
13	2026-03-03 01:34:42.0	sensor 3	acceleration1	0.16725942030512586
14	2026-03-03 01:34:42.0	sensor 3	acceleration2	-1.6562344446105954
15	2026-03-03 01:34:42.0	sensor 3	acceleration3	9.769855958025198
16	2026-03-03 01:34:45.0	sensor 3	acceleration1	1.3473408113146152
17	2026-03-03 01:34:45.0	sensor 3	acceleration2	-1.438300963212904
18	2026-03-03 01:34:45.0	sensor 3	acceleration3	9.728433766935945
19	2026-03-03 01:34:49.0	sensor 3	acceleration1	1.7788905962174844
20	2026-03-03 01:34:49.0	sensor 3	acceleration2	-0.701838824915312
21	2026-03-03 01:34:49.0	sensor 3	acceleration3	9.707459037918971
22	2026-03-03 01:34:52.0	sensor 3	acceleration1	1.010760722794317
23	2026-03-03 01:34:52.0	sensor 3	acceleration2	-0.8957880580372601
24	2026-03-03 01:34:52.0	sensor 3	acceleration3	9.769650615826711
25	2026-03-03 01:34:55.0	sensor 3	acceleration1	0.22181256500064506
26	2026-03-03 01:34:55.0	sensor 3	acceleration2	-0.23279450249857803
27	2026-03-03 01:34:55.0	sensor 3	acceleration3	9.784968173693
28	2026-03-03 01:34:59.0	sensor 3	acceleration1	1.6959710686305403
29	2026-03-03 01:34:59.0	sensor 3	acceleration2	-0.5603152862833356
30	2026-03-03 01:34:59.0	sensor 3	acceleration3	9.737078012729455
31	2026-03-03 01:35:02.0	sensor 3	acceleration1	1.3011296918329027
32	2026-03-03 01:35:02.0	sensor 3	acceleration2	-0.08959232962991392
33	2026-03-03 01:35:02.0	sensor 3	acceleration3	9.82738366928889
34	2026-03-03 01:35:05.0	sensor 3	acceleration1	1.518654866165864
35	2026-03-03 01:35:05.0	sensor 3	acceleration2	-0.9495711748052515
36	2026-03-03 01:35:05.0	sensor 3	acceleration3	9.759894534207806
37	2026-03-03 01:35:09.0	sensor 3	acceleration1	1.0677620867292104
38	2026-03-03 01:35:09.0	sensor 3	acceleration2	-0.31242360224934074
39	2026-03-03 01:35:09.0	sensor 3	acceleration3	9.755870836500849
40	2026-03-03 01:35:12.0	sensor 3	acceleration1	1.903430170134393
41	2026-03-03 01:35:12.0	sensor 3	acceleration2	-1.0919721782503724
42	2026-03-03 01:35:12.0	sensor 3	acceleration3	9.869673718543638
43	2026-03-03 01:35:16.0	sensor 3	acceleration1	0.6273951927718036
44	2026-03-03 01:35:16.0	sensor 3	acceleration2	-1.4915815151801985
45	2026-03-03 01:35:16.0	sensor 3	acceleration3	9.83910695895396
46	2026-03-03 01:35:20.0	sensor 3	acceleration1	0.8928940765876383
47	2026-03-03 01:35:20.0	sensor 3	acceleration2	-0.04173405327795687
48	2026-03-03 01:35:20.0	sensor 3	acceleration3	9.852966791686148
49	2026-03-03 01:35:23.0	sensor 3	acceleration1	0.5006404321190263
50	2026-03-03 01:35:23.0	sensor 3	acceleration2	-0.32429613018287307
51	2026-03-03 01:35:23.0	sensor 3	acceleration3	9.782421487495528

Table: robertclaud_lab6

Columns:

id int AI PK

timestamp timestamp(1)

sensor varchar(45)

measurement varchar(45)

value tinytext

Object Info Session robertclaud_lab6 1 x Apply Revert

Now, let's store the measured data to MySQL database from the sensors (DS18B20 and ADXL345). The schematic of data collection is shown in Figure 6.

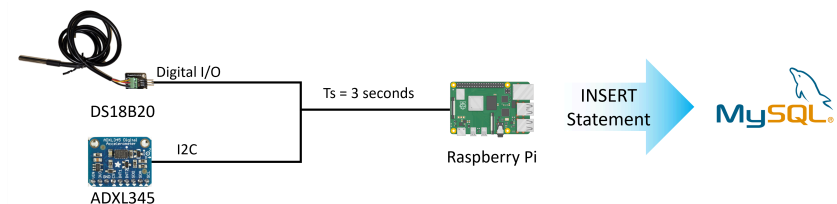


Figure 6 Schematic for Task 2.2

Task 2.2

1. Modify the given sample code (above) to store sensor data (acceleration of each axis from ADXL345, temperature from DS18B20, and humidity from virtual sensor) into the MySQL database you created. The data collection

specifications are below:

- a. Total collection time is 120 seconds.
- b. Sampling period, T_s , is 3 seconds.
- c. Sensor name (sensor column) of ADXL345 is "ADXL345".
- d. Sensor name (sensor column) of DS18B20 is "DS18B20".
- e. Sensor name (sensor column) of Virtual sensor is "Virtual".
- f. Measurements (measurement column) of ADXL345 are "Xacc", "Yacc", "Zacc".
- g. Measurements (measurement column) of ADXL345 are "Temp", "Humd".
- h. Values (value column) of ADXL345 are measured data in unit of m/s^2 .
- i. Values (value column) of DS18B20 are measured data in $^{\circ}C$.
- j. Values (value column) of Virtual sensor are measured data in RH%.

Hint: To measure data, refer to the sample codes ([lab5_ADXL345_sample.py](#) and [lab5_DS18B20_sample.py](#)) in the previous lab.

- 2. Run the modified script.
- 3. Check the stored data in MySQL database using 'MySQL Workbench'.

Task 2.3

- 1. Capture the result grid shown in Figure 5 and attach it to the report.
-

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'robertclaud_lab6' selected. The main window shows a query: `SELECT * FROM ME597Spring260.robertclaud_lab6;` and its results in a grid. The table has 5 columns: id, timestamp, sensor, measurement, and value. The data shows various sensor readings over time.

id	timestamp	sensor	measurement	value
361	2026-03-04 02:16:44.0	DS18B20	Temp	22.5 °C
362	2026-03-04 02:16:44.0	Virtual...	Humd	25.670383820977037 RH%
363	2026-03-04 02:16:48.0	ADXL345	Xacc	-0.196133 m/s2
364	2026-03-04 02:16:48.0	ADXL345	Yacc	0.392266 m/s2
365	2026-03-04 02:16:48.0	ADXL345	Zacc	9.5320638 m/s2
366	2026-03-04 02:16:48.0	DS18B20	Temp	22.5 °C
367	2026-03-04 02:16:48.0	Virtual...	Humd	32.991002731498 RH%
368	2026-03-04 02:16:52.0	ADXL345	Xacc	-0.2353596 m/s2
369	2026-03-04 02:16:52.0	ADXL345	Yacc	0.392266 m/s2
370	2026-03-04 02:16:52.0	ADXL345	Zacc	9.571290399999999 m/s2
371	2026-03-04 02:16:52.0	DS18B20	Temp	24.875 °C
372	2026-03-04 02:16:52.0	Virtual...	Humd	48.75549060717466 RH%
373	2026-03-04 02:16:56.0	ADXL345	Xacc	-0.2353596 m/s2
374	2026-03-04 02:16:56.0	ADXL345	Yacc	0.3530394 m/s2
375	2026-03-04 02:16:56.0	ADXL345	Zacc	9.571290399999999 m/s2
376	2026-03-04 02:16:56.0	DS18B20	Temp	24.875 °C
377	2026-03-04 02:16:56.0	Virtual...	Humd	26.55977037386431 RH%
378	2026-03-04 02:17:01.0	ADXL345	Xacc	-0.196133 m/s2
379	2026-03-04 02:17:01.0	ADXL345	Yacc	0.3530394 m/s2
380	2026-03-04 02:17:01.0	ADXL345	Zacc	9.492837199999999 m/s2
381	2026-03-04 02:17:01.0	DS18B20	Temp	28.187 °C
382	2026-03-04 02:17:01.0	Virtual...	Humd	29.674571466264787 RH%
383	2026-03-04 02:17:05.0	ADXL345	Xacc	-0.196133 m/s2
384	2026-03-04 02:17:05.0	ADXL345	Yacc	0.392266 m/s2
385	2026-03-04 02:17:05.0	ADXL345	Zacc	9.5320638 m/s2
386	2026-03-04 02:17:05.0	DS18B20	Temp	29.937 °C
387	2026-03-04 02:17:05.0	Virtual...	Humd	28.13140864072342 RH%
388	2026-03-04 02:17:09.0	ADXL345	Xacc	-0.2353596 m/s2
389	2026-03-04 02:17:09.0	ADXL345	Yacc	0.392266 m/s2
390	2026-03-04 02:17:09.0	ADXL345	Zacc	9.5320638 m/s2
391	2026-03-04 02:17:09.0	DS18B20	Temp	31.812 °C
392	2026-03-04 02:17:09.0	Virtual...	Humd	10.39859727948218 RH%
393	2026-03-04 02:17:09.0	ADXL345	Xacc	-0.196133 m/s2
394	2026-03-04 02:17:13.0	ADXL345	Yacc	0.3530394 m/s2
395	2026-03-04 02:17:13.0	ADXL345	Zacc	9.4536106 m/s2
396	2026-03-04 02:17:13.0	DS18B20	Temp	33.562 °C
397	2026-03-04 02:17:13.0	Virtual...	Humd	35.14697611695493 RH%
398	2026-03-04 02:17:17.0	ADXL345	Xacc	0.0392266 m/s2
399	2026-03-04 02:17:17.0	ADXL345	Yacc	0.4707192 m/s2
400	2026-03-04 02:17:17.0	ADXL345	Zacc	8.316039199999999 m/s2
401	2026-03-04 02:17:17.0	DS18B20	Temp	35.25 °C
402	2026-03-04 02:17:17.0	Virtual...	Humd	95.54830191939155 RH%
403	2026-03-04 02:17:21.0	ADXL345	Xacc	-0.2353596 m/s2
404	2026-03-04 02:17:21.0	ADXL345	Yacc	0.4707192 m/s2
405	2026-03-04 02:17:21.0	ADXL345	Zacc	9.5320638 m/s2
406	2026-03-04 02:17:21.0	DS18B20	Temp	36.875 °C
407	2026-03-04 02:17:21.0	Virtual...	Humd	0.9536663180342186 RH%
408	2026-03-04 02:17:26.0	ADXL345	Xacc	-0.5491724 m/s2
409	2026-03-04 02:17:26.0	ADXL345	Yacc	0.7060788 m/s2
410	2026-03-04 02:17:26.0	ADXL345	Zacc	10.9049948 m/s2
411	2026-03-04 02:17:26.0	DS18B20	Temp	38.375 °C
412	2026-03-04 02:17:26.0	Virtual...	Humd	13.4629162549456 RH%
413	2026-03-04 02:17:30.0	ADXL345	Xacc	-1.1767979999999998 m/s2
414	2026-03-04 02:17:30.0	ADXL345	Yacc	0.3138128 m/s2
415	2026-03-04 02:17:30.0	ADXL345	Zacc	11.6110736 m/s2
416	2026-03-04 02:17:30.0	DS18B20	Temp	39.875 °C
417	2026-03-04 02:17:30.0	Virtual...	Humd	27.128659110273112 RH%

2. Place the modified code below

```
In [ ]: # Paste your code below.
#

import datetime
import time
import random
import mysql.connector # Using mysql.connector since it is the official MySQL driver for Python and is widely used.

# From Lab5_DS18B20_sample.py
import os
import glob

# Taken from Lab5_ADXL345_sample.py
import board
import busio
import adafruit_adxl34x
import datetime
```

```

class DS18B20:
    def __init__(self):
        self.sensor = "DS18B20"
        os.system('modprobe w1-gpio')
        os.system('modprobe w1-therm')
        self.base_dir = '/sys/bus/w1/devices/'
        self.device_folder = glob.glob(self.base_dir + '28*')[0]
        self.device_file = self.device_folder + '/w1_slave'

    def read_temp_raw(self):
        f = open(self.device_file, 'r')
        lines = f.readlines()
        f.close()
        return lines

    def read_temp(self):
        lines = self.read_temp_raw()
        while lines[0].strip()[-3:] != 'YES':
            time.sleep(0.2)
            lines = self.read_temp_raw()
        equals_pos = lines[1].find('t=')
        if equals_pos != -1:
            temp_string = lines[1][equals_pos+2:]
            temp_c = float(temp_string) / 1000.0
            return temp_c

    def to_db(self, cursor, timestamp):
        temp_c = self.read_temp()
        params = {
            'table': TABLE,
            'timestamp': timestamp,
            'sensor': self.sensor,
            'measurement': 'Temp',
            'value': str(temp_c) + " °C"
        }
        cursor.execute(QUERY, params)
        print("{}={}".format(params['measurement'],params['value']))

class ADXL345:
    def __init__(self):
        self.sensor = "ADXL345"
        self.i2c = busio.I2C(board.SCL, board.SDA)
        self.acc = adafruit_adxl34x.ADXL345(self.i2c)

    def read_acceleration(self):
        x_acc, y_acc, z_acc = self.acc.acceleration
        return x_acc, y_acc, z_acc

    def to_db(self, cursor, timestamp):
        x_acc, y_acc, z_acc = self.read_acceleration()
        params = {
            'table': TABLE,
            'timestamp': timestamp,
            'sensor': self.sensor,
        }

        # X Acceleration
        params['measurement'] = 'Xacc'
        params['value'] = str(x_acc) + " m/s2"
        cursor.execute(QUERY, params)
        print("{}={}".format(params['measurement'],params['value']))

        # Y Acceleration
        params['measurement'] = 'Yacc'
        params['value'] = str(y_acc) + " m/s2"
        cursor.execute(QUERY, params)
        print("{}={}".format(params['measurement'],params['value']))

        # Z Acceleration

```

```

        params['measurement'] = 'Zacc'
        params['value'] = str(z_acc) + " m/s2"
        cursor.execute(QUERY, params)
        print("{}={}".format(params['measurement'],params['value']))

class VirtualSensor:
    def __init__(self):
        self.sensor = "VirtualSensor"

    def to_db(self, cursor, timestamp):
        value = random.uniform(0, 100)
        params = {
            'table': TABLE,
            'timestamp': timestamp,
            'sensor': self.sensor,
            'measurement': 'Humd',
            'value': str(value) + " RH%"
        }
        cursor.execute(QUERY, params)
        print("{}={}".format(params['measurement'],params['value']))

## Credential
HOST = '10.165.67.146' # MySQL server host DNS
PORT = 3306 # MySQL server port number
USER = 'robertclaud' # MySQL account name
PASSWORD = 'Qwer1234!' # Password of the account
DB = 'ME597Spring260' # DB name
TABLE = 'robertclaud_lab6' # table name
QUERY = f"""
    INSERT INTO {TABLE} (
        timestamp,
        sensor,
        measurement,
        value
    ) VALUES (
        %(timestamp)s,
        %(sensor)s,
        %(measurement)s,
        %(value)s
    );
"""

connection = mysql.connector.connect(
    host=HOST,
    user=USER,
    password=PASSWORD,
    db=DB,
    port=PORT
) # make a connection to MySQL server

cursor = connection.cursor(dictionary=True) # Open cursur to execute SQL query

# Set to 120 per (a)
duration = 120 # collection time in second unit
start_time = time.time() # get current time

while time.time() - start_time < duration:
    timestamp = datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S') # get datetime as MySQL timestamp form

    acc_sensor = ADXL345()
    acc_sensor.to_db(cursor, timestamp)

    temp_sensor = DS18B20()
    temp_sensor.to_db(cursor, timestamp)

    virtual_sensor = VirtualSensor()
    virtual_sensor.to_db(cursor, timestamp)
    connection.commit()
    print("==INSERT QUERIES DONE==\n")

```

```
time.sleep(3) # wait for 3 seconds per (b)

connection.close() # gently close connection
print("==Program DONE==")

#
```

Lab6 Summary and Deliverables

Answer the following questions for your achievements

Q1. Please summarize Lab6.

In Lab 6, we learned the basics of SQL and databases. We learned how to create new tables in a MySQL instance, insert new data into it, and retrieve it using SQL queries. We then connected an ADXL345 accelerometer, a DS18B20 thermocouple, and a virtual sensor to our raspberry pi. Using this, we were able to populate the table we created with real sensor readings.

Q2. What skills did you have to develop to accomplish this project?

To accomplish this project I developed better handling of the SQL syntax and language. Although I have used SQL before, I had never used MySQL and needed to learn how to parameterize the queries. I continued to improve my Python skills and in particular utilized object oriented programming.

Q3. What aspects of this project were the most beneficial for your learning?

The aspects of this project that were the most beneficial for my learning were creating a table by myself and populating it with real-world data that I was able to observe. I had a candle nearby so I was able to watch the temperature rise on the thermocouple readings and see this change in the database.

Q4. What challenges did you encounter in completing the project?

In completing this project, I had difficulty pushing my data to the database for a bit as I was unfamiliar with the syntax of pymysql. In the past, I have used Oracle and PostgreSQL databases. Additionally I typically use SQLAlchemy as my module of choice since it has better cross-compatibility support with other python packages like pandas.

Q5. How did you overcome the challenges or remedy the problems encountered?

To overcome these challenges I decided to switch to using the mysql-connector-python module so that I could parameterize my queries using built-in dictionaries. The syntax is quite similar to pymysql so it was not much of a learning curve. In addition, I had forgotten to include cursor.commit() during my first few runs, so when I added the line the data populated correctly.

Get back to [Lab Index Page](#)